Akshay Pranav - Project Portfolio Page

Overview

ItemTasker is a keyboard-first warehouse inventory management system built for speed. Register SKUs to a 3x3 physical grid, attach maintenance and operational tasks with due dates and priorities, then sort, filter, and mark them off — all from the terminal. Data is persisted automatically on exit and reloaded seamlessly on the next launch.

Summary of Contributions

Code contributed

RepoSense link

Enhancements implemented

1. Edit SKU (`editsku`)

What it does: Allows users to relocate an existing SKU to a new warehouse grid cell without affecting any of its attached tasks.

Justification: Warehouse inventory is dynamic — pallets and stock are physically moved regularly. Without this command, a manager would have to delete and recreate a SKU (losing all task history) just to update a location.

Highlights:

1. Implemented full validation chain in `SKUCommandHandler#handleEditSku()`: null-checks, case-insensitive SKU lookup via `CommandHelper`, and `Location` enum parsing with rejection of invalid grid cells
2. Used in-place mutation via `SKU#setLocation()` so all tasks remain attached to the object — no risk of orphaned data
3. Added `java.util.logging` at every validation stage (missing args, SKU not found, invalid location, successful update) to aid debugging

Code: `SKUCommandHandler#handleEditSku`, `SKU#setLocation`

2. Edit Task (`edittask`)

What it does: Updates any combination of a task's due date, priority, and description in a single command. Only the fields explicitly provided are changed — all other fields are preserved.

Justification: Requiring users to re-enter unchanged fields is error-prone and slow. The partial-update design avoids accidental overwrites and keeps commands concise.

Highlights:

1. Implemented a 7-stage validation chain: required argument checks → at-least- one-field guard → date validation via `DateValidator` → index parsing → SKU lookup → bounds check → priority parsing. Each

stage returns early on failure with a specific error message
2. Integrated `DateValidator.validateDateOrError()` to catch both malformed formats and impossible calendar dates (e.g. `2026-02-30`) before any mutation
3. Delegates to `SKUTaskList#editSKUTask()` which applies only non-null fields, reinforcing the partial-update contract at the model level

Code: `TaskCommandHandler#handleEditTask`, `SKUTaskList#editSKUTask`, `DateValidator`

3. Mark / Unmark Task (`marktask`, `unmarktask`)

What it does: Toggles a task's completion status. Both commands guard against idempotent operations — if the task is already in the target state, the user receives an informational message instead of a misleading success message.

Justification: Without the idempotency check, marking an already-done task prints "[OK]" with no actual change, which erodes trust in the system's feedback.

Highlights:

1. Both handlers share the same validation structure: index parsing → SKU lookup → bounds checking → idempotency guard → delegation to `SKUTaskList#markTask()` / `unmarkTask()`
2. Assertion guards in `SKUTask#mark()` and `SKUTask#unmark()` enforce correct pre-conditions at the model level — the command layer is responsible for verifying state before calling these methods
3. Added logging at mark/unmark delegation points for traceability

Code: `TaskCommandHandler#handleMarkTask`, `TaskCommandHandler#handleUnmarkTask`, `SKUTaskList#markTask`, `SKUTaskList#unmarkTask`, `SKUTask#mark`, `SKUTask#unmark`

4. Sort Tasks (`sorttasks`)

What it does: Sorts a SKU's task list by `date`, `priority`, or `status` in ascending or descending order.

Justification: Managers need to triage tasks quickly. Sorting by priority surfaces urgent work; sorting by date surfaces overdue items. Without sorting, users must mentally re-order long task lists themselves.

Highlights:

1. Designed and implemented `TaskSorter` as a standalone, SRP-compliant class. It takes a defensive copy of the task list (original list unmodified), builds a type-safe `Comparator` via a switch on the sort field, and applies reversal for descending order
2. `TaskSorter.isValidSortField()` is called in the handler before instantiation, preventing invalid comparator construction from reaching the model
3. `TaskSorter` constructor calls `sort()` internally — the sorted result is immediately available via `getSortedTasks()` with no additional setup

Code: `TaskCommandHandler#handleSortTask`, `TaskSorter`

Also implemented `DateValidator`, a shared static utility used by both `addskutask` and `edittask` for strict `YYYY-MM-DD` date validation — two-stage check (regex format match, then `LocalDate.parse()` to catch impossible calendar dates like Feb 30). Reused across the codebase by other handlers as well.

Contributions to the User Guide

1. **FAQ section**: Wrote 1 question covering multi-field task editing with `edittask`
2. **Command Summary table**: Full cheat sheet of all 16 commands with format and examples

Contributions to the Developer Guide

1. **Design > Architecture section**: Wrote the overall architecture description with the architecture diagram explaining the component structure (UI → Parser → CommandRunner → Handlers → Model → Storage)
2. **Edit SKU / Edit Task implementation section**: Step-by-step scenario walkthroughs, sequence and architecture diagrams, and design considerations (in-place mutation vs delete-and-recreate; wrapper pattern vs direct object access)

UML diagrams contributed:

- `architecture.puml` (overall architecture)
- `edit-sku-sequence.puml`, `edit-sku-architecture.puml`
- `edit-task-sequence.puml`, `edit-task-architecture.puml`
- `markTaskSequence.puml`, `mark-unmark-architecture.puml`, `unmarkTaskSequence.puml`
- `sort-task-sequence.puml`, `sort-task-architecture.puml`

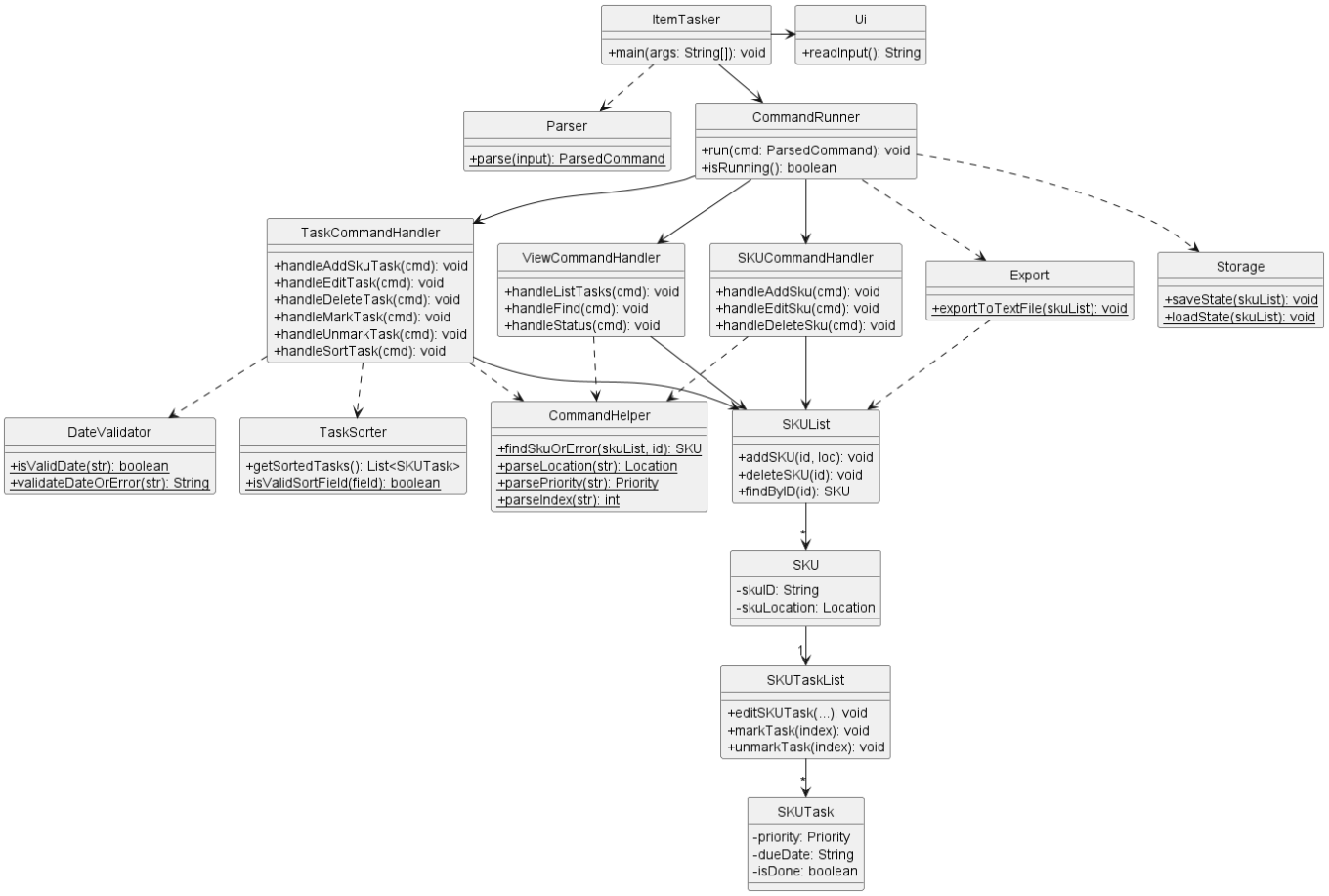
Contributions to team-based tasks

1. **CommandRunner refactor (PR #100)**: Split the monolithic `CommandRunner` into `TaskCommandHandler`, `SKUCommandHandler`, and `ViewCommandHandler`, applying SRP to improve maintainability for the whole team
2. Set up and cleaned up `AboutUs.md` entry (PRs #17, #20)
3. Fixed `listtasks` empty-SKU message (PR #76, Issue #75)
4. Resolved merge conflicts in `DeveloperGuide.md` (PRs #145, #146)
5. Updated `architecture.puml` after CommandRunner refactor to keep DG consistent with codebase (PR #163)

Contributions to the Developer Guide (Extracts)

Design > Architecture

ItemTasker follows a layered architecture with clear separation of concerns:



- **Ui** handles all user interface interactions
- **Parser** transforms raw input into structured `ParsedCommand` objects
- **CommandRunner** dispatches commands to the appropriate handler
- **SKUCommandHandler** manages SKU-level commands (`addsku`, `editsku`, `deletesku`)
- **TaskCommandHandler** manages task-level commands (`addskutask`, `edittask`, `deletetask`, `marktask`, `unmarktask`, `sorttasks`)
- **ViewCommandHandler** manages read-only commands (`listtasks`, `find`, `status`)
- **CommandHelper** and **DateValidator** provide shared validation utilities
- **TaskSorter** sorts tasks by date, priority, or completion status

Note: Solid arrows (→) indicate direct dependencies or composition. Dashed arrows (- ->) indicate utility dependencies (e.g. static helper calls). The "" multiplicity on model relationships denotes one-to-many.

Key Design Principles:

- **Single Responsibility**: Each handler class owns exactly one category of commands
- **Command Delegation**: `CommandRunner` acts purely as a dispatcher with no business logic
- **Encapsulation**: Each `SKU` owns its own `SKUTaskList` — no external maps or redundant data structures
- **Layered Architecture**: UI → Logic → Model → Storage separation

Edit SKU / Edit Task Feature

Edit SKU

Given below is an example usage scenario for the Edit SKU mechanism.

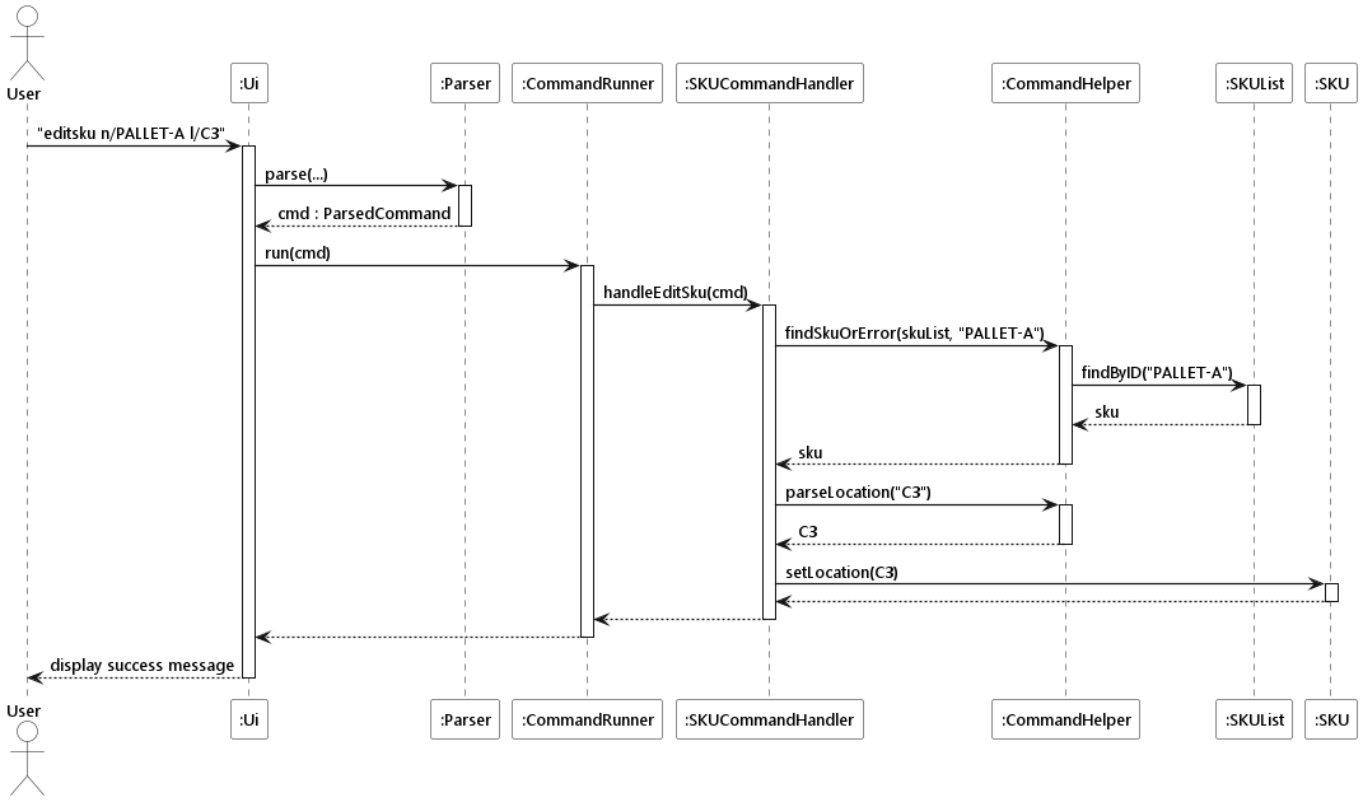
Step 1. The user executes `editsku n/PALLET-A l/C3`. The `Ui` reads the input, and the `Parser` maps the arguments into a `ParsedCommand` object.

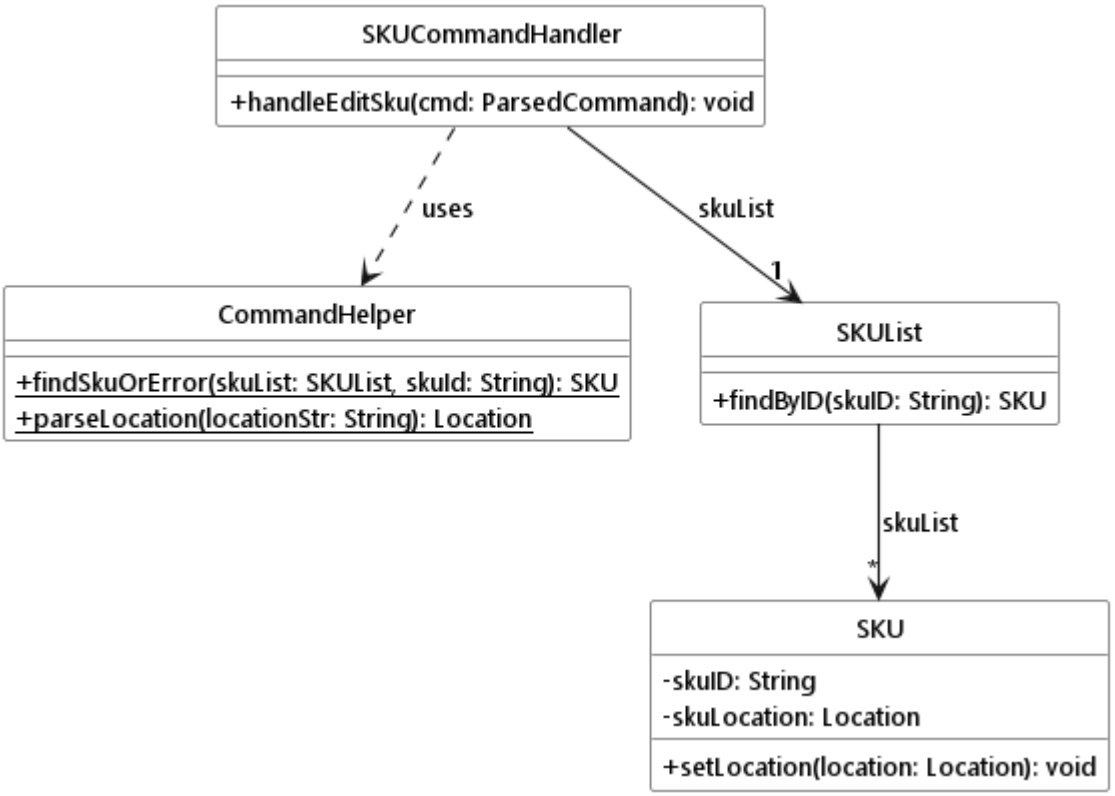
Step 2. The `CommandRunner#run()` method routes execution to `SKUCommandHandler#handleEditSku()`.

Step 3. `handleEditSku()` calls `CommandHelper.findSkuOrError()` to locate the target `SKU` via case-insensitive lookup. If not found, an error is printed and the method returns early.

Step 4. The handler calls `CommandHelper.parseLocation("C3")` to validate and convert the string into a `Location` enum. If the location is invalid (e.g. `Z9`), an error is printed and the method returns.

Step 5. `SKU#setLocation(Location.C3)` is called, updating the SKU's location in place. All existing tasks attached to the SKU are preserved.





Edit Task

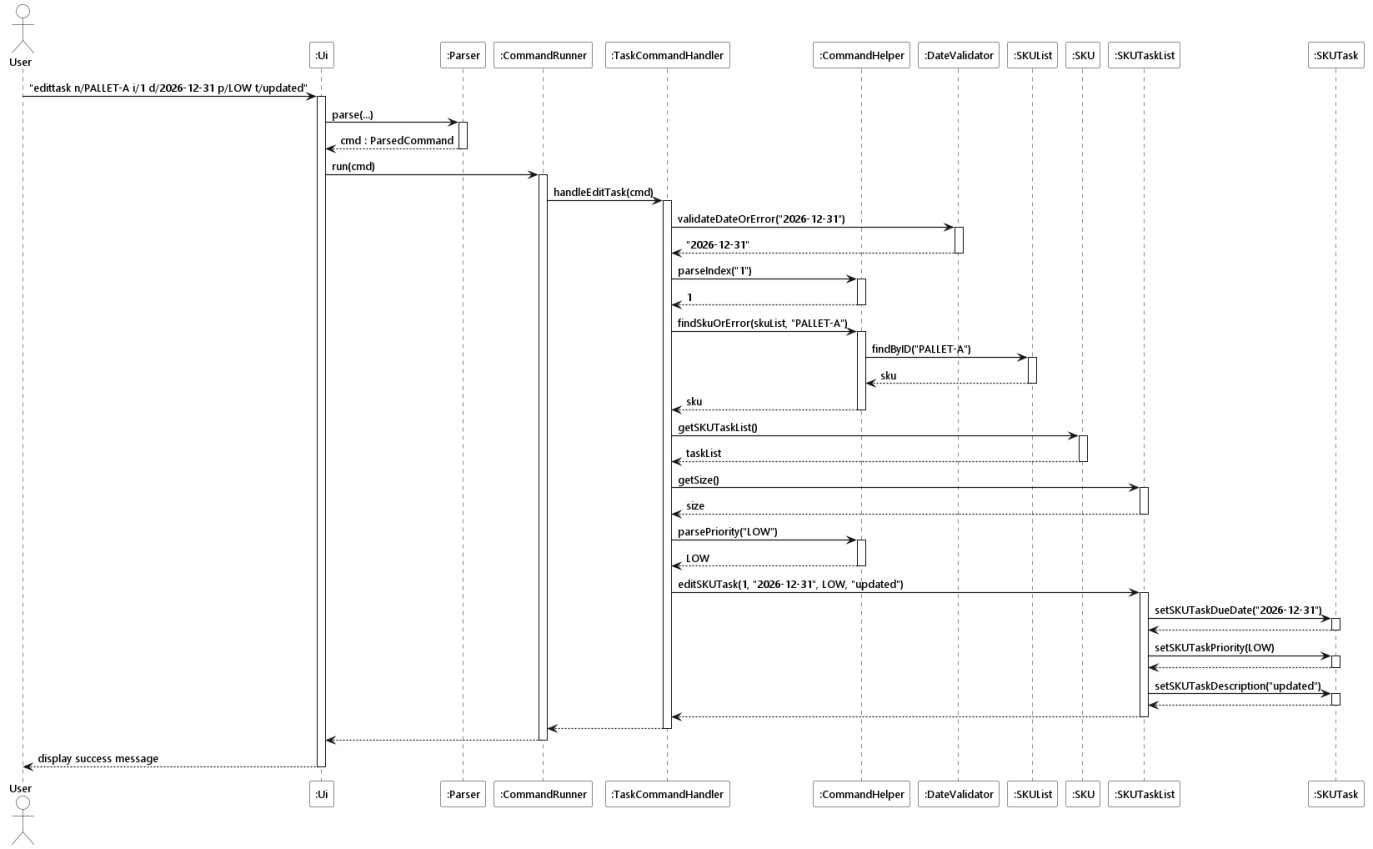
Step 1. The user executes `edittask n/PALLET-A i/1 d/2026-12-31 p/LOW t/updated`.

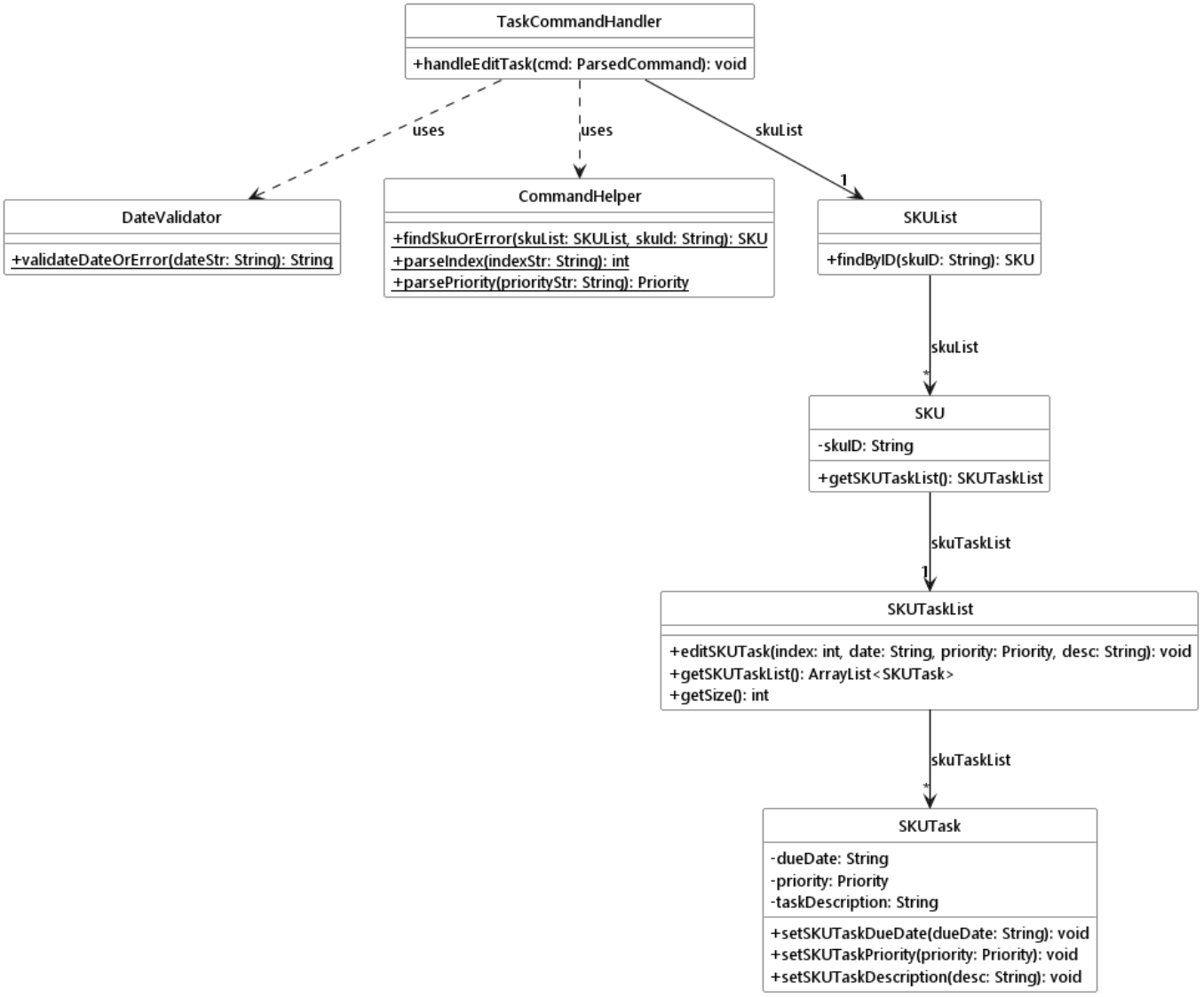
Step 2. The `CommandRunner#run()` method routes execution to `TaskCommandHandler#handleEditTask()`.

Step 3. `handleEditTask()` performs a multi-stage validation chain:

1. Checks required arguments (`n/` SKU ID, `i/` task index) are present
2. Checks at least one editable field (`d/`, `p/`, `t/`) is provided
3. Validates date format via `DateValidator.validateDateOrError()`
4. Parses and validates index via `CommandHelper.parseIndex()`
5. Locates SKU via `CommandHelper.findSkuOrError()`
6. Bounds-checks the index against the task list size
7. Parses and validates priority via `CommandHelper.parsePriority()`

Step 4. Only after all validations pass, the handler calls `SKUTaskList#editSKUTask()`. Unchanged fields are preserved.


User
:Ui
:Parser
:CommandRunner
:TaskCommandHandler
:CommandHelper
:DateValidator
:SKUList
:SKU
:SKUTaskList
:SKUTask
"edittask n/PALLET-A i/1 d/2026-12-31 p/LOW t/updated"
parse(...)
cmd : ParsedCommand
run(cmd)
handleEditTask(cmd)
validateDateOrError("2026-12-31")
"2026-12-31"
parseIndex("1")
1
findSkuOrError(skuList, "PALLET-A")
findByID("PALLET-A")
sku
sku
getSKUTaskList()
taskList
getSize()
size
parsePriority("LOW")
LOW
editSKUTask(1, "2026-12-31", LOW, "updated")
setSKUTaskDueDate("2026-12-31")
setSKUTaskPriority(LOW)
setSKUTaskDescription("updated")
display success message


TaskCommandHandler
+handleEditTask(cmd: ParsedCommand): void
uses
uses
skuList
1
DateValidator
+validateDateOrError(dateStr: String): String
CommandHelper
+findSkuOrError(skuList: SKUList, skuId: String): SKU
+parseIndex(indexStr: String): int
+parsePriority(priorityStr: String): Priority
SKUList
+findByID(skuID: String): SKU
skuList
*
SKU
-skuID: String
+getSKUTaskList(): SKUTaskList
skuTaskList
1
SKUTaskList
+editSKUTask(index: int, date: String, priority: Priority, desc: String): void
+getSKUTaskList(): ArrayList<SKUTask>
+getSize(): int
skuTaskList
*
SKUTask
-dueDate: String
-priority: Priority
-taskDescription: String
+setSKUTaskDueDate(dueDate: String): void
+setSKUTaskPriority(priority: Priority): void
+setSKUTaskDescription(desc: String): void

Contributions to the User Guide (Extracts)

FAQ

Q: Can I edit multiple fields of a task at once? **A**: Yes. The `edittask` command accepts any combination of `d/` (due date), `p/` (priority), and `t/` (description) flags simultaneously. Only the fields you specify will be updated. All other fields are preserved.

Command Summary

Action	Format	Example
Add SKU	`addsku n/SKU_ID l/LOCATION`	`addsku n/PALLET-A l/B2`
Edit SKU	`editsku n/SKU_ID l/NEW_LOCATION`	`editsku n/PALLET-A l/C3`
Delete SKU	`deletesku n/SKU_ID`	`deletesku n/PALLET-A`
Add task	`addskutask n/SKU_ID d/DUE_DATE [p/PRIORITY] [t/DESCRIPTION]`	`addskutask n/PALLET-A d/2026-06-15 p/HIGH t/Check stock`
Edit task	`edittask n/SKU_ID i/TASK_INDEX [d/DUE_DATE] [p/PRIORITY] [t/DESCRIPTION]`	`edittask n/PALLET-A i/1 d/2026-12-31 p/LOW`
Delete task	`deletetask n/SKU_ID i/TASK_INDEX`	`deletetask n/PALLET-A i/2`
Mark task	`marktask n/SKU_ID i/TASK_INDEX`	`marktask n/PALLET-A i/1`
Unmark task	`unmarktask n/SKU_ID i/TASK_INDEX`	`unmarktask n/PALLET-A i/1`
Sort tasks	`sorttasks n/SKU_ID s/SORT_FIELD [o/ORDER]`	`sorttasks n/PALLET-A s/priority o/descending`
List tasks	`listtasks [n/SKU_ID] [p/PRIORITY] [l/LOCATION]`	`listtasks p/HIGH`
Find tasks	`find [n/SKU_ID] [t/DESCRIPTION] [i/TASK_INDEX]`	`find n/PALLET-A t/Check`
View status	`status [n/SKU_ID]`	`status n/PALLET-A`
View warehouse map	`viewmap`	`viewmap`
Export inventory	`export`	`export`
Help	`help`	`help`
Exit	`bye` / `exit`	`bye`